

Softwares

Usually for Linux, there should be a package manager available to install new softwares. However, as we don't have the permission for such tasks, we have to rely on other methods to install softwares.

Module System

The module system used by UPPMAX is called Lmod, but for convenience, it is wrapped as the module command in UPPMAX. You can find all available modules [here](#)¹.

- To **search a module**, use `$ module spider <MODULE_NAME>`

Tip

You can search a module with RegExp with the `-r` option as:

```
$ module -r spider <EXPRESSION>
```

- To **load a module**, use `$ module add <MODULE_NAME>`

Note

The system distinguishes the upper cases and the lower cases, which means that when you want to load a module called FFmpeg, it will complain if you write ffmepg.

- To **unload all modules**, use `$ module purge`
- To **list loaded modules**, use `$ module list`

Installing Binary

However, there are still chances that a module is unavailable on the system. While you can send an email to UPPMAX support, you can download the binary and run it locally.

Compiling

It is a rare situation, but if there is no binary file provided, you can try to compile things yourself.

There is no a common method to build a software, as there are so many toolchains for different usages and different languages. Thus, this session is more like a reminder that this way exists, and you can follow the provided guide (if any) to build the software.

¹This table list softwares on *Bianca*, but all of them should be included in *Pelle* also.

Building

Configure the build using CMake:

```
cmake -Bbuild -DCMAKE_INSTALL_PREFIX=/usr
```

NOTE: The `-DCMAKE_INSTALL_PREFIX` option above will install the files to the `/usr` directory, instead of the default `/usr/local` location. For more information on available options, you can run:

```
cmake -Bbuild -LH
```

To use a different compiler, or compiler flags, you can specify these before the `cmake` command. For example:

```
CC=clang CFLAGS=-Wextra cmake -Bbuild -DCMAKE_INSTALL_PREFIX=/usr
```

The `espeak-ng` and `speak-ng` programs, along with the `espeak-ng` voices, can then be built with:

```
cmake --build build
```

The data (language dictionaries, phoneme tables, intonation) can be built with:

```
cmake --build build --target data
```

Specific languages can be compiled by running the built binary directly:

```
ESPEAK_DATA_PATH=`pwd`/build build/src/espeak-ng --compile=LANG
```

Figure 1: I once had to compile the software myself following the provided guide.